Cooperative optimization - Numerical project :
Cooperative Kernel regression

Lucien Le Gall, Noé Casteleyn, Loris Manganelli

March 31, 2026

The GitHub repository of our project (in particular the notebook) can be found at:

https://github.com/Loris-Manganelli/projet-cooperative-optimisation/tree/main

1 Part I : Data visualization and first 4 algorithms

1.1 Data visualization

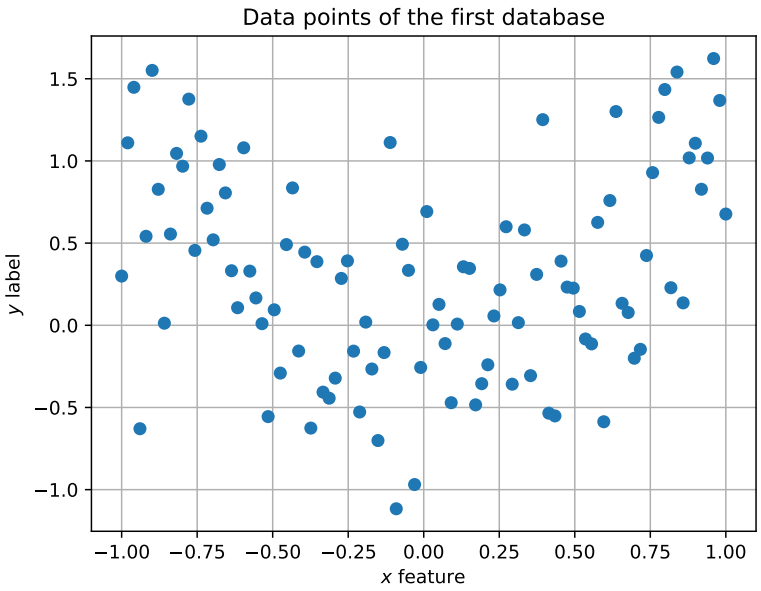

Figure 1: Visualization of the first database for the first 100 data points

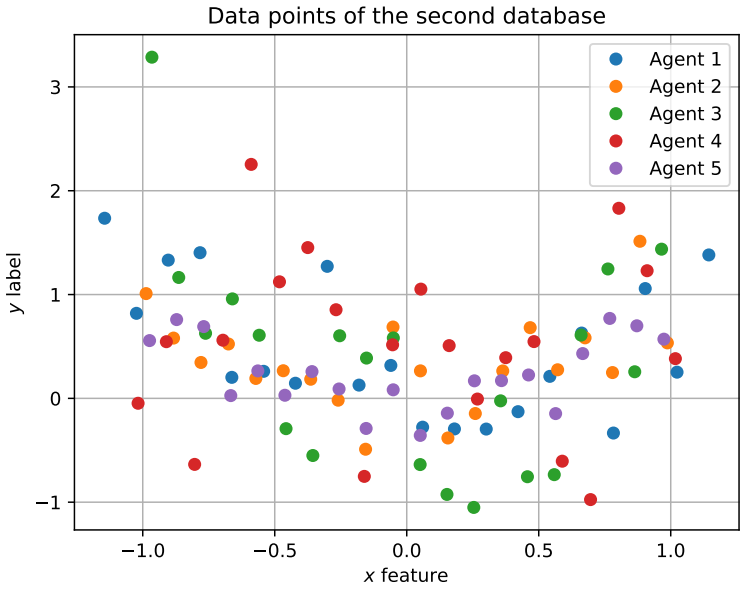

Figure 2: Visualization of the second database for the first 100 data points

1.2 Decentralized gradient descent and gradient tracking

1.2.1 Justification of the chosen step size

Let us denote the agent $A_1, A_2, \ldots, A_5$ (it also denotes the data points of the dataset belonging to this agent). The function f_i for each agent is:

$$f_i(\alpha) = \frac{\sigma^2}{5}\frac{1}{2}\alpha^T K_{mm}\alpha + \frac{1}{2}\sum_{j\in A_i}(y_j - K_{(j)m}\alpha)^2 + \frac{\nu}{10}\|\alpha\|^2 \tag{1}$$

One has:

$$\nabla_\alpha f_i(\alpha) = \frac{\sigma^2}{5}K_{mm}\alpha - \sum_{j\in A_i} K^T_{(j)m}(y_i - K_{(i)m}\alpha) + \frac{\nu}{5}\alpha \tag{2}$$

Then:

$$\nabla^2_\alpha f_i(\alpha) = \frac{\sigma^2}{5}K_{mm} + \sum_{j\in A_i} K^T_{(j)m}K_{(j)m} + \frac{\nu}{5}I_m \tag{3}$$

So, let $\lambda_{i,\max} = \underset{\lambda\in \mathrm{Sp}\left(\nabla^2_\alpha f_i(\alpha)\right)}{\arg\max} |\lambda|$ for each agent i. Let $L = \max_i |\lambda_{i,\max}|$. Then, we need to choose a step size which fulfilled is $\leq O\left(\frac{1}{L}\right)$. We have computed it using Python, and found $\frac{1}{L} \approx 0.010$. That is to say in the following $s \leq 10^{-2}$

1.2.2 Convergence plots depending on graph structure

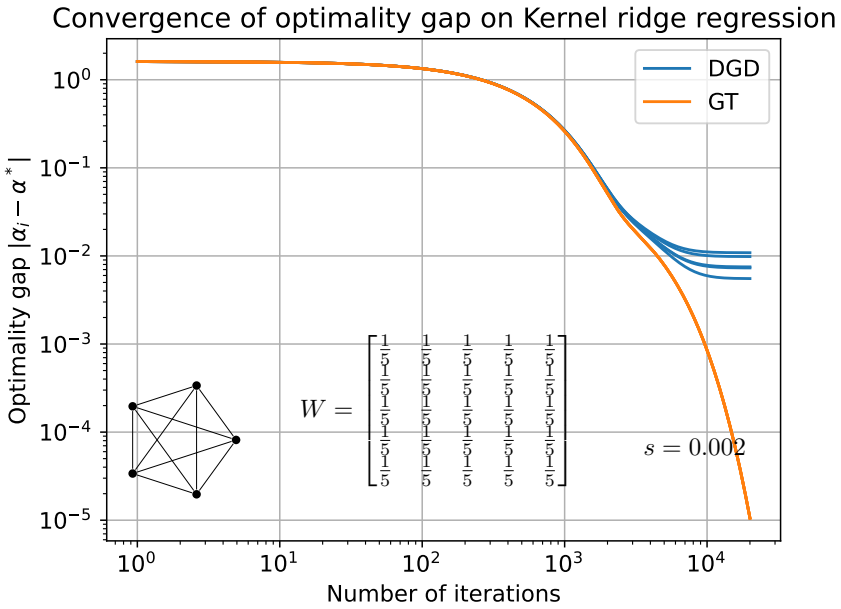

Figure 3: Convergence of DGD and Gradient Tracking, in terms of $\|\alpha_i - \alpha^*\|$, on the Kernel ridge regression for a fully connected graph.

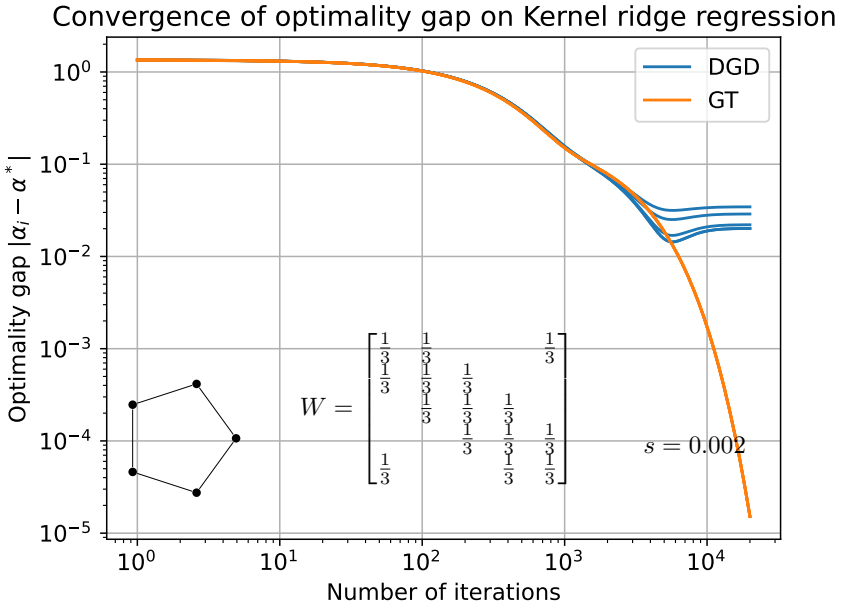

Figure 4: Convergence of DGD and Gradient Tracking, in terms of $\|\alpha_i - \alpha^*\|$, on the Kernel ridge regression for a circular graph.

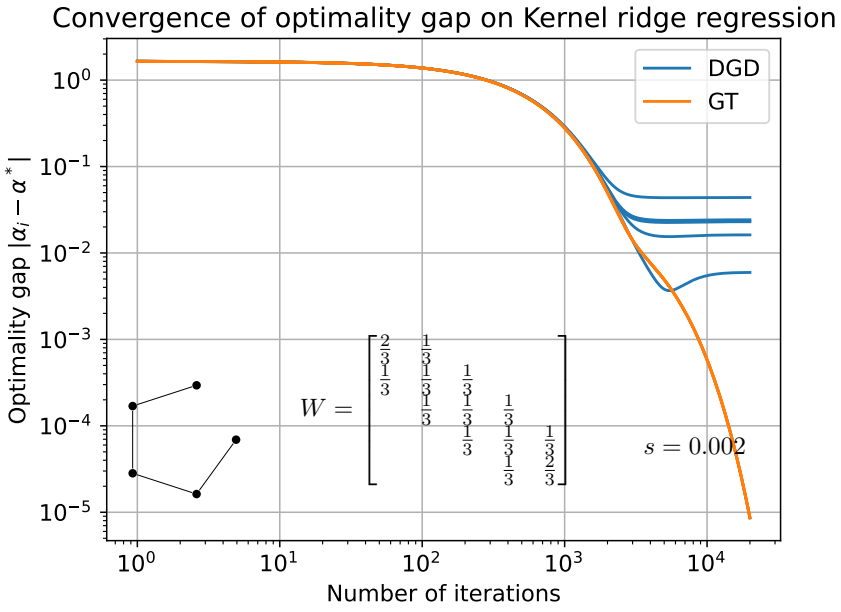

Figure 5: Convergence of DGD and Gradient Tracking, in terms of $\|\alpha_i - \alpha^*\|$, on the Kernel ridge regression for a line graph.

Comments. Concerning the DGD, we observe that the smaller the error ball, the better is the consensus. Indeed, one has theoretically that the error ball for DGD is $O\left(\frac{\alpha}{1-\gamma}\right)$ where $\frac{1}{1-\gamma}$ is the spectral gap of the graph, and α is the step size. We could reach zero error but slower, with a diminishing step size. We also have an error on the consensus, which is of order $O\left(\frac{\alpha}{1-\gamma}\right)$. The gradient tracking algorithm reaches zero error ball and zero consensus error.

1.2.3 Visualization of the obtained function

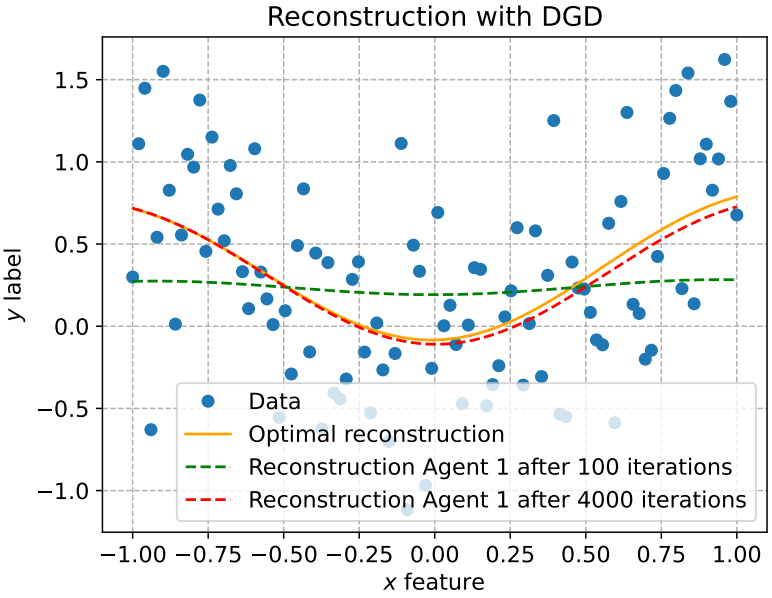

Figure 6: Obtained function with DGD, for different number of iterations and comparison with the optimum

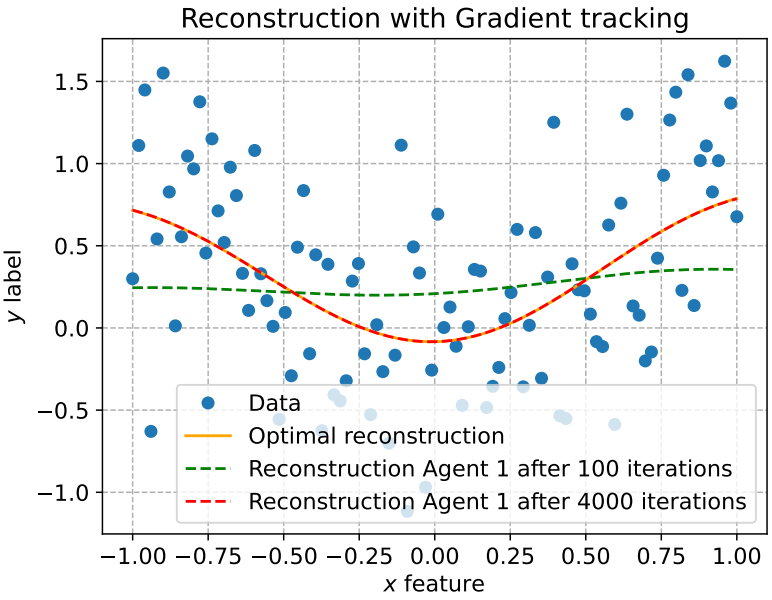

Figure 7: Obtained function with Gradient Tracking, for different number of iterations and comparison with the optimum

Comments. We clearly notice that the reconstruction after 100 iterations for the first agent is better for the gradient tracking method than for DGD. Moreover, after 4000 iterations, the reconstruction is quite satisfying for both methods meaning that we get really close to the optimum. The results are even better for ADMM and dual decomposition in terms of reconstruction (*cf* notebook).

1.3 Dual Methods (Dual decomposition and ADMM)

We denote F the following function:

$$F(y) = \sum_{i=1} f_i(x^i) \quad s.t. \quad Ay = 0$$

Where $y = (x^i)_{1 \leq i \leq 5}$, And $A = I \otimes Id_m$, with I the incidence matrix of the graph, where each edge is arbitrarily oriented.

1.3.1 Justification of the stepzise

The dual problem is $\sigma^2_{max}(I)/m$ -Smooth. Where $m = \max(m_i)$ is the strong convexity constant of F. In our case we find : `step_size` $= O(10^{-2})$ typically `step_size` $= 0.01$ is chosen.

1.3.2 Convergence plots

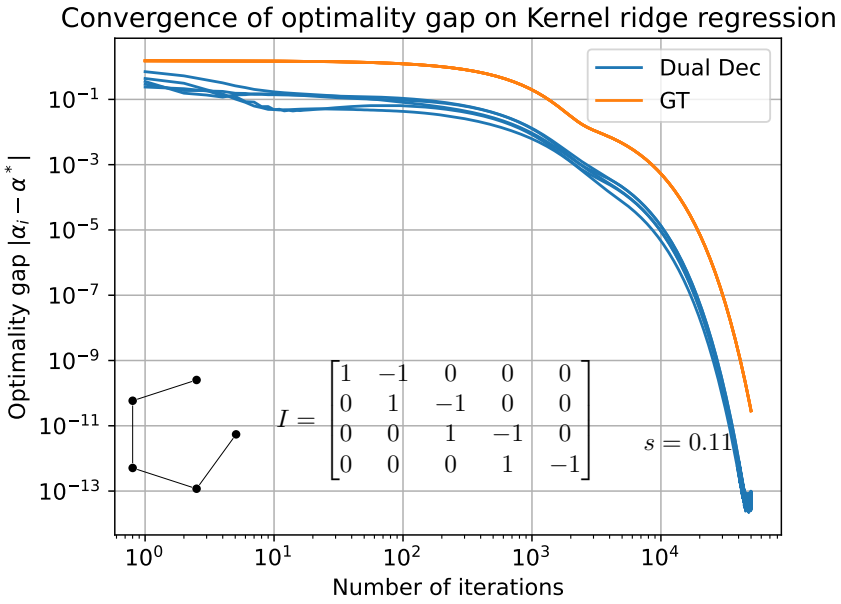

Figure 8: Convergence of Dual Decomposition and Gradient tracking, in terms of $\|\alpha_i - \alpha^*\|$ on the Kernel ridge regression for a line graph. The step size s is for the dual decomposition only, I is the oriented incidence matrix.

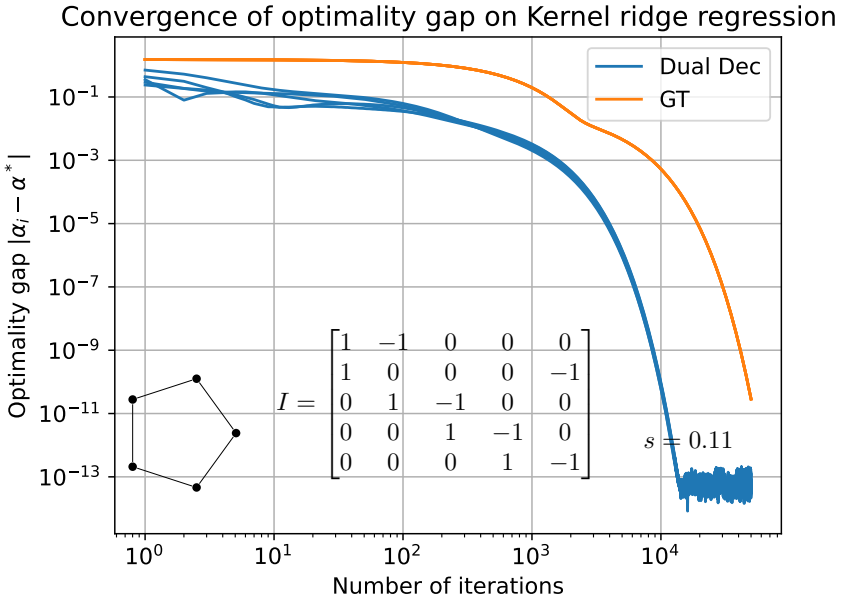

Figure 9: Convergence of Dual Decomposition and Gradient tracking, in terms of $\|\alpha_i - \alpha^*\|$ on the Kernel ridge regression for a circular graph. The step size s is for the dual decomposition only, I is the oriented incidence matrix.

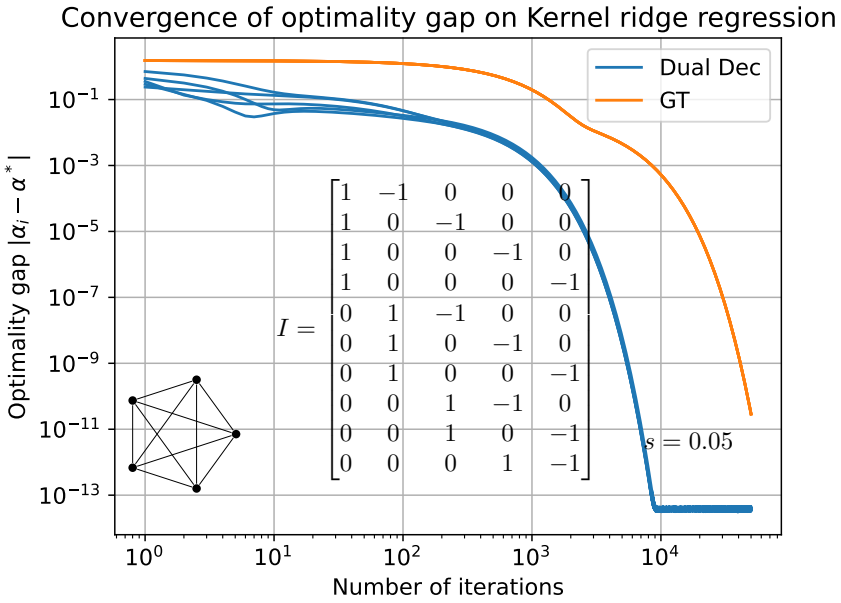

Figure 10: Convergence of Dual Decomposition and Gradient tracking, in terms of $\|\alpha_i - \alpha^*\|$ on the Kernel ridge regression for a full graph. The step size s is for the dual decomposition only, I is the oriented incidence matrix.

Dual decomposition In the first two examples, a step size of $s = 0.11$ is chosen, at the limit $2m/\sigma^2_{max}(I)$. However, for the fully connected graph, $\sigma^2_{max}(I)$ is twice bigger, so the algorithm doesn't converge anymore.

When the graph is more connected, each solution converges faster to a consensus.

ADMM For ADMM the main parameter to play with is the β parameter of the augmented Lagrangian.

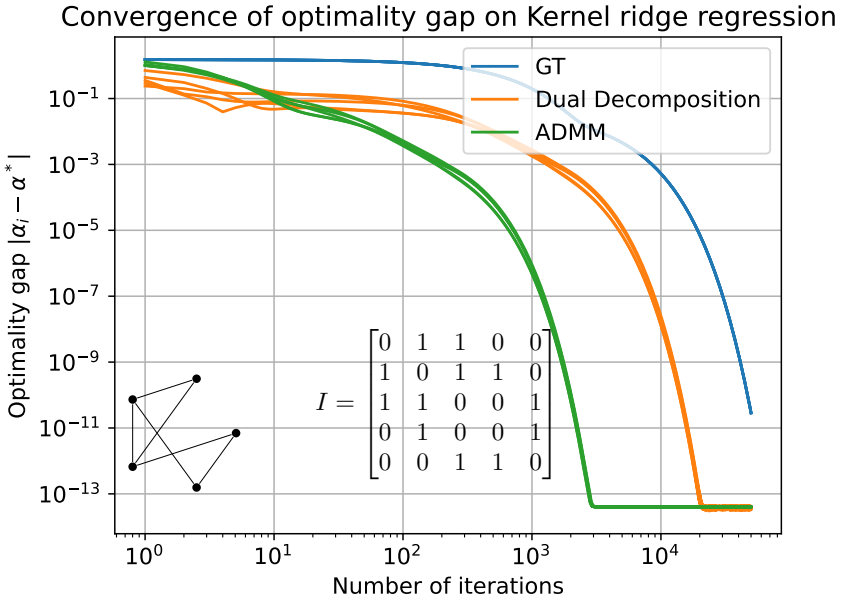

Figure 11: Convergence of ADMM ($\beta = 1$), Dual Decomposition (step size $= 0.07$), and Gradient tracking (stepsize$= 0.02$), in terms of $\|\alpha_i - \alpha^*\|$ on the Kernel ridge regression.

We see that ADMM is faster than Dual decomposition and Gradient Tracking with step sizes

chosen optimally. We chose $\beta = 1$ because of the next result:

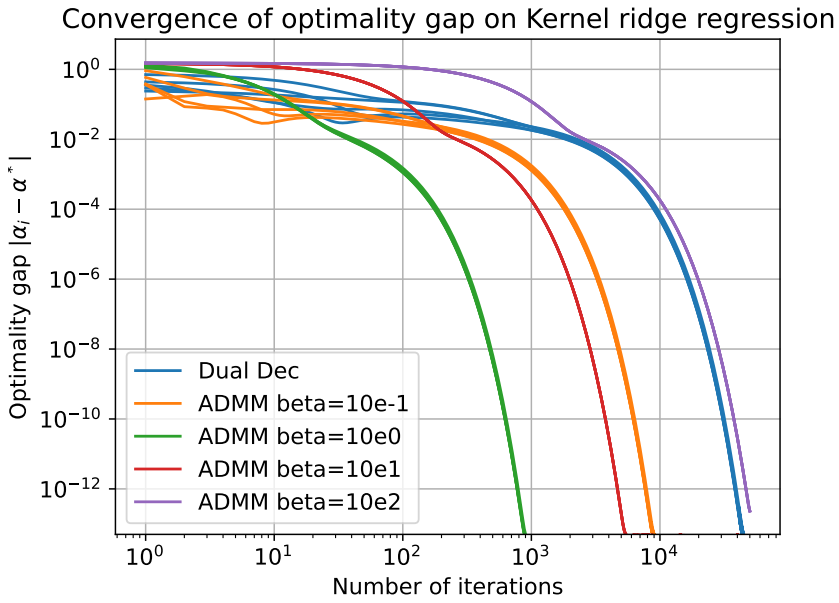

Figure 12: Convergence of ADMM ($\beta = 1$) for different β constant, and Dual Decomposition (step size = 0.07), in terms of $\|\alpha_i - \alpha^*\|$ on the Kernel ridge regression for a full graph.

1.4 Increasing n

Our results on the different methods when increasing n, while using $m \approx \sqrt{n}$, are the following one (we used adaptive step size to counter effect the diminishing smoothness constant).

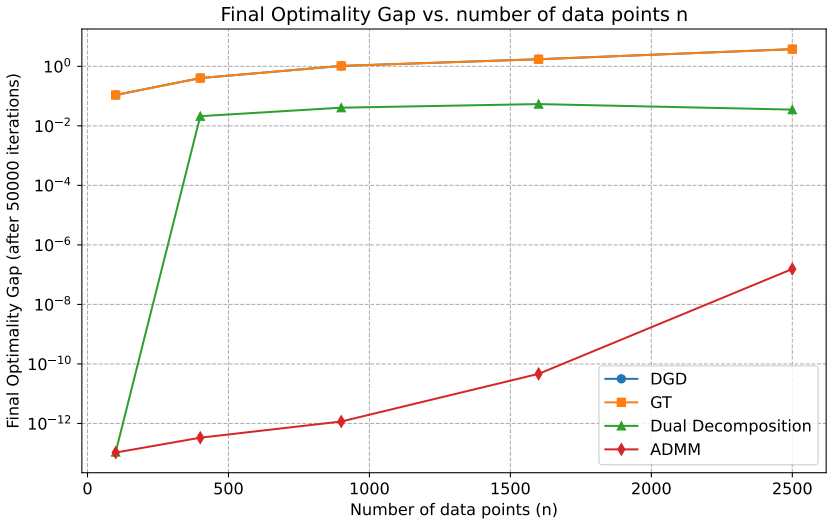

Figure 13: Final gap when increasing n for the different methods

We measure the gap as the average over agent of final optimally gap. We notice that the gradient in the term 2 is higher (because we sum more terms), so the term $\frac{1}{L}$ decreases resulting in slower convergence.

2 Part II

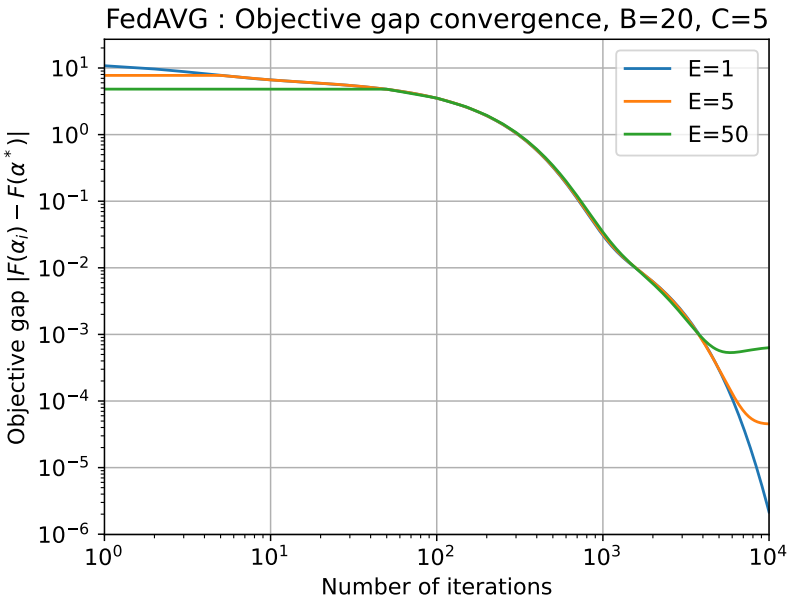

Figure 14: Convergence of FedAVG in terms of $\|F(\alpha_i) - F(\alpha^*)\|$ on the Kernel Ridge Regression example, for a constant learning rate of 0.002, and varying the number of epochs E

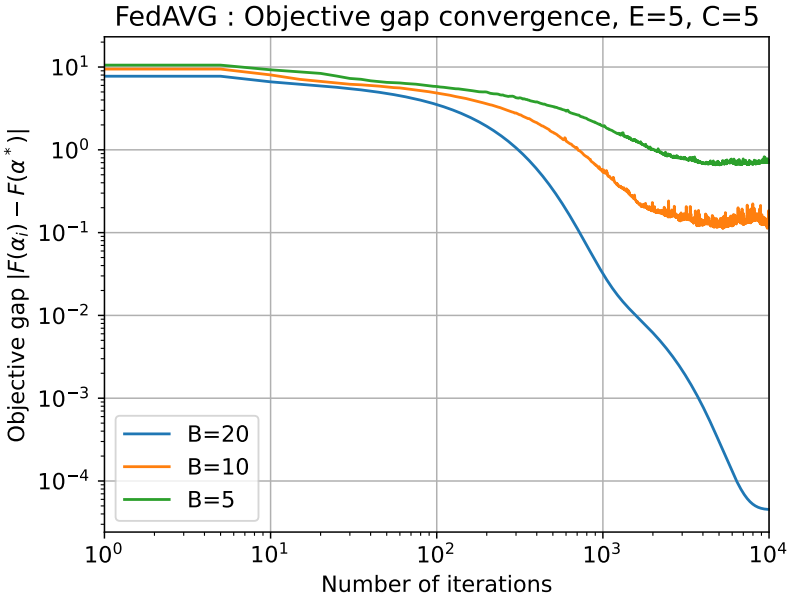

Figure 15: Convergence of FedAVG in terms of $\|F(\alpha_i) - F(\alpha^*)\|$ on the Kernel Ridge Regression example, for a constant learning rate of 0.002, and varying the batch size B

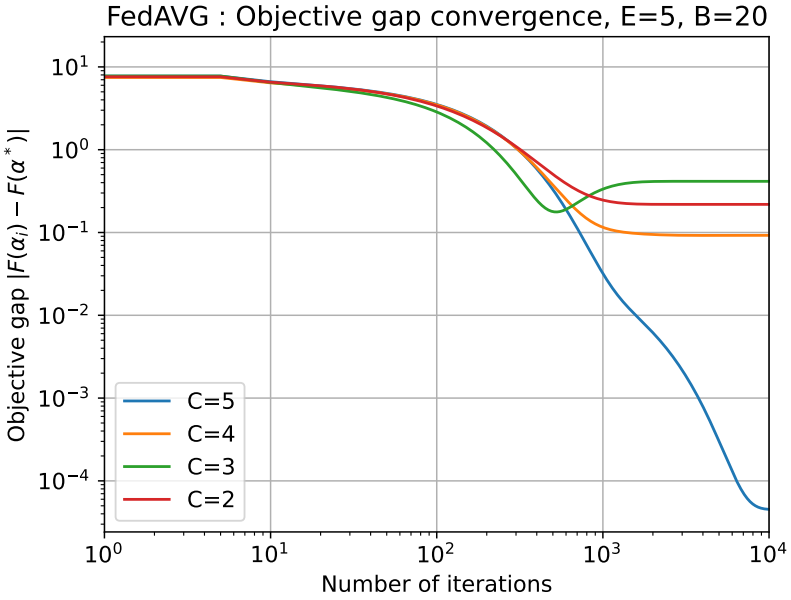

Figure 16: Convergence of FedAVG in terms of $\|F(\alpha_i) - F(\alpha^*)\|$ on the Kernel Ridge Regression example, for a constant learning rate of 0.002, and varying the number of devices C

Comments. In the case $B = 20$ (full-batch case) and $E = 1$, we can observe that the FedAVG algorithm boils down to DGD. That is why, on figure 14, we observe an error that goes to 0 asymptotically. Results observed on figures 15 are expected, since reducing the batch-size brings us closer to a stochastic gradient algorithm, which is known for stopping to improve sooner than the full-batch setting. Finally,, figure 16 is also pretty intuitive, since using less data obviously leads to less qualitative results. It is however surprising to note that using $C = 2$ devices leads to better results than using $C = 3$.

3 Part III

We notice that we obtained convergence for $\epsilon = 10$, and $\epsilon = 1$. For $\epsilon = 10$, the performance is degraded, although the theoretical guarantees still hold.

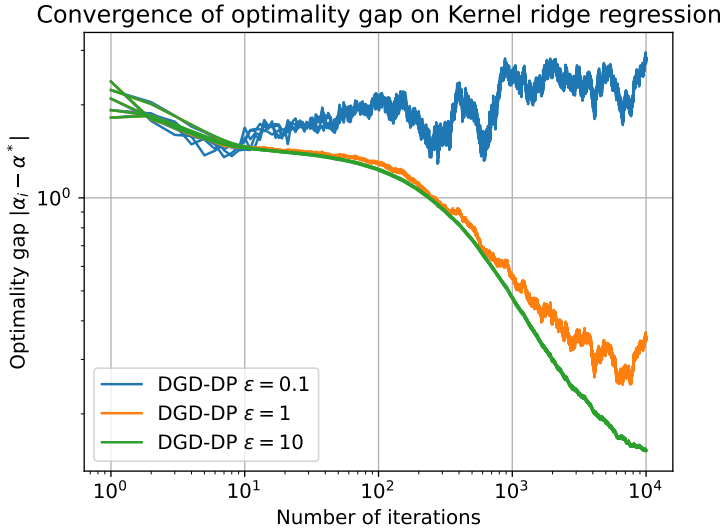

Figure 17: Convergence of DGD-DP in terms of $\|\alpha_i - \alpha^*\|$ for a line graph